

HOME AUTOMATION USING ARDUINO

Bachelor of Engineering Degree in Electrical and Communication in
Engineering(ECE)

By

Pasupuleti Pavan Kumar

Hindu College of Engineering and Technology

ABSTRACT

The main objective of this project is to develop a home automation system using an Arduino board with Bluetooth being remotely controlled by any Android OS smart phone. As technology is advancing so houses are also getting smarter. Modern houses are gradually shifting from conventional switches to centralized control system, involving remote controlled switches. Presently, conventional wall switches located in different parts of the house makes it difficult for the user to go near them to operate. Even more it becomes more difficult for the elderly or physically handicapped people to do so. Remote controlled home automation system provides a most modern solution with smart phones. In order to achieve this, a Bluetooth module is interfaced to the Arduino board at the receiver end while on the transmitter end, a GUI application on the cellphone sends ON/OFF commands to the receiver where loads are connected. By touching the specified location on the GUI, the loads can be turned ON/OFF remotely through this technology. The loads are operated by Arduino board through opto-isolators and thyristors using TRIACs.

CONTENTS

BONAFIDE...	2
DECLARATION.....	3
ACKNOWLEDGEMENT	4
ABSTRACT	5
CONTENTS.....	6
LISTOFFIGURES... ..	7
INTRODUCTION	8
COMPONENTSREQUIRED... ..	9
DESCRIPTION	10
BLOCKDIAGRAM	40
CIRCUITDIAGRAM	41
CODE... ..	43
APPLICATION.....	44
CONCLUSION	47
REFERENCE	48

INTRODUCTION

Nowdays, we have remote controls for our television set sand other electronicsystems, which have made our lives real easy. Have you ever wondered about home automation which would givethefacilityofcontrollingtubelights,fansandotherelectricalappliancesathomeusinga remotecontrol?Off-course, Yes!But,aretheavailableoptionscost-effective?Iftheansweris No, we have found a solution to it. We have come up with a new system called Arduino based homeautomationusingBluetooth. Thissystemissuper-costeffectiveandcangivetheuser,the abilitytocontrolanyelectronicdevicewithoutevenspendingforaremotecontrol. Thisproject helps the user to control all the electronic devices using his/her smartphone. Time is a very valuablething. Everybodywantstosavetimeasmuchasthey can. Newtechnologiesarebeing introduced to save our time. To save people's time we are introducing Home Automation system using Bluetooth . With the help of this system you can control your home appliances from your mobile phone. You can turn on/off your home appliances within the range of Bluetooth.

COMPONENTS REQUIRED

- 1) ARDUINOUNO
- 2) 1-CHANNELRELAY(5v)
- 3) BLUETOOTH MODULEHC05
- 4) POWERSUPPLY
- 5) LOAD(BULB220V)
- 6) SWITCH BOARD
- 7) CONNECTINGWIRES
- 8) SMARTPHONE(BLUETOOTHENABLED)

DESCRIPTION

ARDUINO UNO

Arduino is an open source computer hardware and software company, project, and user community that designs and manufactures single-board microcontrollers and microcontroller kits for building digital devices and interactive objects that can sense and control objects in the physical and digital world. The project's products are distributed as open-source hardware and software, which are licensed under the GNU LesserGeneral Public License (LGPL) or the GNU General Public License (GPL), permitting the manufacture of are Arduino boards and software distribution by anyone. arduino boards availablecommerciallyinpreassembledform,orasdo-it-yourself(DIY)kits.

Arduino board designs use a variety of microprocessors and controllers. The boards are equipped with sets of digital and analog input/output (I/O) pins that may be interfaced to various expansion board sor Breadboards(shields) and other circuits.The boards feature serial communications interfaces, including Universal Serial Bus (USB) on some models, which are also used for loading programs from personal computers. The microcontrollers are typically programmed using a dialect of features from the programming languages C and C++. In additiontousingtraditionalcompilertoolchains,theArduinoprojectprovidesanintegrateddevelopmentenvironment(IDE)basedontheProcessinglanguageproject.

TheArduinoprojectstartedin2003asaprogramforstudentsattheInteractionDesignInstituteIvreainIvrea, Italy, aiming to provide a low- costand easy way for novices and professionals to create devices that interact with their environment using sensors and actuators. Common examples of such devices intended for beginner hobbyists include simple robots, thermostats, and motiondetectors.

Thename**Arduino**comesfromabarinIvrea,Italy,wheresomeofthefoundersoftheproject usedtomeet.ThebarwasnamedafterArduinooofIvrea,whowasthemargraveoftheMarchofIvreaandKingofItalyfrom1002to1014.

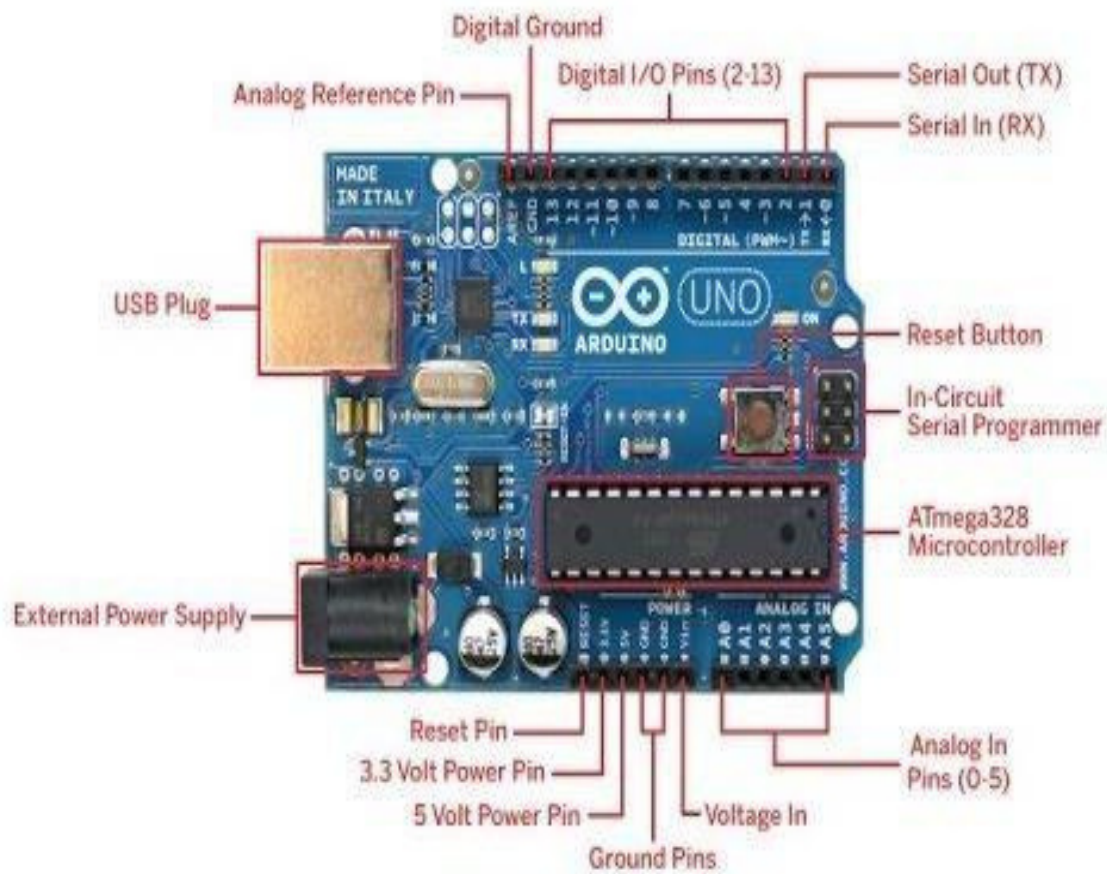


FIG 1 ARDUINO UNO R3

ARDUINO HARDWARE PART:-

Arduino is open-source hardware. The hardware reference designs are distributed under a Creative Commons Attribution Share-Alike 2.5 license and are available on the Arduino website. Layout and production files for some versions of the hardware are also available.

Although the hardware and software designs are freely available under copyleft licenses, the developers have requested the name **Arduino** to be exclusive to the official product and not be used for derived works without permission. The official policy document on use of the Arduino emphasizes that the project is open to incorporating work by others into the official product. Several Arduino-compatible products commercially released have avoided the project name by using various names ending in **-duino**.

Most Arduino boards consist of an Atmel 8-bit AVR microcontroller (ATmega8, ATmega168, ATmega328, ATmega1280, ATmega2560) with varying amounts of flash memory, pins, and features. The 32-bit Arduino Due, based on the Atmel SAM3X8E was introduced in 2012. The boards use single or double-row pins or female headers that facilitate connections for programming and incorporation into other circuits. These may connect with add-on modules termed **shields**. Multiple and possibly stacked shields may be individually addressable via an I²C serial bus. Most boards include a 5 V linear regulator and a 16 MHz crystal oscillator or ceramic resonator. Some designs, such as the LilyPad, run at 8 MHz and dispense with the on-board voltage regulator due to specific form-factor restrictions.

Arduino microcontrollers are pre-programmed with a boot loader that simplifies uploading of programs to the on-chip flash memory. The default bootloader of the Arduino UNO is the optiboot bootloader. Boards are loaded with program code via a serial connection to another computer. Some serial Arduino boards contain a level shifter circuit to convert between RS-232 logic levels and transistor–transistor logic (TTL) level signals. Current Arduino boards are programmed via Universal Serial Bus (USB), implemented using USB-to-serial adapter chips such as the FTDI FT232. Some boards, such as later-model Uno boards, substitute the FTDI chip with a separate AVR chip containing USB-to-serial firmware, which is reprogrammable via its own ICSP header. Other variants, such as the Arduino Mini and the unofficial Boarduino, use a detachable USB-to-serial adapter board or cable, Bluetooth or other methods. When used with

traditional microcontroller tools, instead of the Arduino IDE, standard AVR in-system programming (ISP) programming is used.

The Arduino board exposes most of the microcontroller's I/O pins for use by other circuits. The *Diecimila*, *Duemilanove*, and current *Uno* provide 14 digital I/O pins, six of which can produce pulse-width modulated signals, and six analog inputs, which can also be used as six digital I/O pins. These pins are on the top of the board, via female 0.1-inch (2.54mm) headers. Several plug-in application shields are also commercially available. The Arduino Nano, and Arduino-compatible Bare Bones Board and Arduino boards may provide male header pins on the underside of the board that can plug into solderless breadboards.

Many Arduino-compatible and Arduino-derived boards exist. Some are functionally equivalent to an Arduino and can be used interchangeably. Many enhance the basic Arduino by adding output drivers, often for use in school-level education, to simplify making buggies and small robots. Others are electrically equivalent but change the form factor, sometimes retaining compatibility with shields, sometimes not. Some variants use different processors, of varying compatibility.

ARDUINO SOFTWARE PART:-

IDE

The Arduino integrated development environment (IDE) is a cross-platform application (for Windows, macOS, Linux) that is written in the programming language Java. It originated from the IDE for the languages **Processing** and **Wiring**. It includes a code editor with features such as text cutting and pasting, searching and replacing text, automatic indenting, brace matching, and syntax highlighting, and provides simple **one-click** mechanisms to compile upload programs to an Arduino board. It also contains a message area, a text console, a toolbar and with buttons for common functions and a hierarchy of operation menus. The source code for the IDE is released under the GNU General Public License, version 2.

The Arduino IDE supports the languages C and C++ using special rules of code structuring. The Arduino IDE supplies a software library from the Wiring project, which provides many common input and output procedures. User-written code only requires two basic functions, for starting the sketch and the main program loop, that are compiled and linked with a program stub **main()** into an executable cyclic executive program with the GNU toolchain, also included with the IDE distribution. The Arduino IDE employs the program **avrdude** to convert the executable code into a text file in hexadecimal encoding that is loaded into the Arduino board by a loader program in the board's firmware.

Sketch

A program written with the Arduino IDE is called a **sketch**. [58] Sketches are saved on the development computer as text files with the file extension **.ino**. Arduino Software (IDE) pre-1.0 saved sketches with the extension **.pde**.

A minimal Arduino C/C++ program consists of only two functions:

setup(): This function is called once when a sketch starts after power-up or reset. It is used to initialize variables, input and output pin modes, and other libraries needed in the sketch.

loop(): After **setup()** has been called, function **loop()** is executed repeatedly in the main program. It controls the board until the board is powered off or is reset.

Blink example

Most Arduino boards contain a light-emitting diode (LED) and a load resistor connected between pin 13 and ground, which is a convenient feature for many tests and program functions. A typical program for a beginning Arduino programmer blinks a LED repeatedly. This program uses the functions **pinMode()**, **digitalWrite()**, and **delay()**, which are provided by the internal libraries included in the IDE environment. This program is usually loaded into a new Arduino board by the manufacturer.

APPLICATIONS OF RELAY:-

Relays are used wherever it is necessary to control a high power or high voltage circuit with a low power circuit, especially when galvanic isolation is desirable. The first application of relays was in long telegraph lines, where the weak signal received at an intermediate station could control a contact, regenerating the signal for further transmission. High-voltage or high-current devices can be controlled with small, low voltage wiring and pilot switches. Operators can be isolated from the high voltage circuit. Low power devices such as microprocessors can drive relay allows the high current of the cranking motor to be controlled with small wiring and contacts in the ignition key. relays to control electrical loads beyond their direct drive capability. In an automobile, a starter Electromechanical switching systems including Strowger and Crossbar telephone exchanges made extensive use of relays in ancillary control circuits. The Relay Automatic Telephone Company also manufactured telephone exchanges based solely on relay switching techniques designed by Gottlieb Ansgarius Betulander. The first public relay based telephone exchange in the UK was installed in Fleetwood on 15 July 1922 and remained in service until 1959.

The use of relays for the logical control of complex switching systems like telephone exchanges was studied by Claude Shannon, who formalized the application of Boolean algebra to relay circuit design in *A Symbolic Analysis of Relay and Switching Circuits*. Relays can perform the basic operations of Boolean combinatorial logic. For example, the boolean AND function is realised by connecting normally open relay contacts in series, the OR function by connecting closed

contact. Relays were used for control of automated systems for machine tools and production lines. The Ladder programming language is often used for designing relay logic networks.

normally open contacts in parallel. Inversion of a logical input can be done with a normally

Early electro-mechanical computers such as the ARRA, Harvard Mark II, Zuse Z2, and Zuse Z3 used relays for logic and working registers. However, electronic devices proved faster and easier to use.

Because relays are much more resistant than semiconductors to nuclear radiation, they are widely used in safety-critical logic, such as the control panels of radioactive waste-handling machinery. Electromechanical protective relays are used to detect overload and other faults on electrical lines by opening and closing circuit breakers.

SINGLE CHANNEL RELAY MODULE

Overview

We can control high voltage electronic devices using relays. A Relay is actually a switch which is electrically operated by an electromagnet. The electromagnet is activated with a low voltage, for example 5 volts from a microcontroller and it pulls a contact to make or break a high voltage circuit.

FIG 2 CHANNEL RELAY



Single-Channel Relay Module Specifications

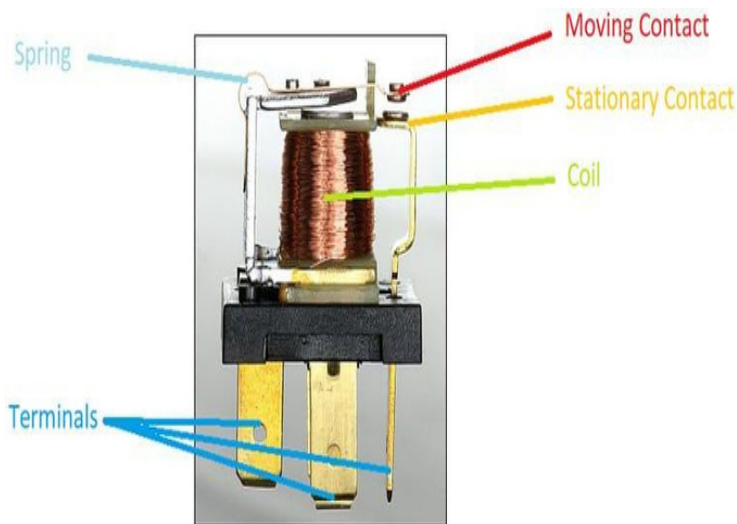
Supply voltage – 3.75V to 6V
Quiescent current: 2mA
Current when the relay is active: ~70mA
Relay maximum contact voltage – 250VAC or 30VDC
Relay maximum current – 10A

SINGLECHANNEL RELAY MODULE PIN DESCRIPTION

<i>PIN NUMBER</i>	PIN NAME	DESCRIPTION
1	Relay Trigger	Input to activate the relay

2	Ground	0V reference
3	VCC	Supply input for powering the relay coil
4	Normally Open	Normally open terminal of the relay
5	Common	Common terminal of the relay
6	Normally Closed	Normally closed contact of the relay

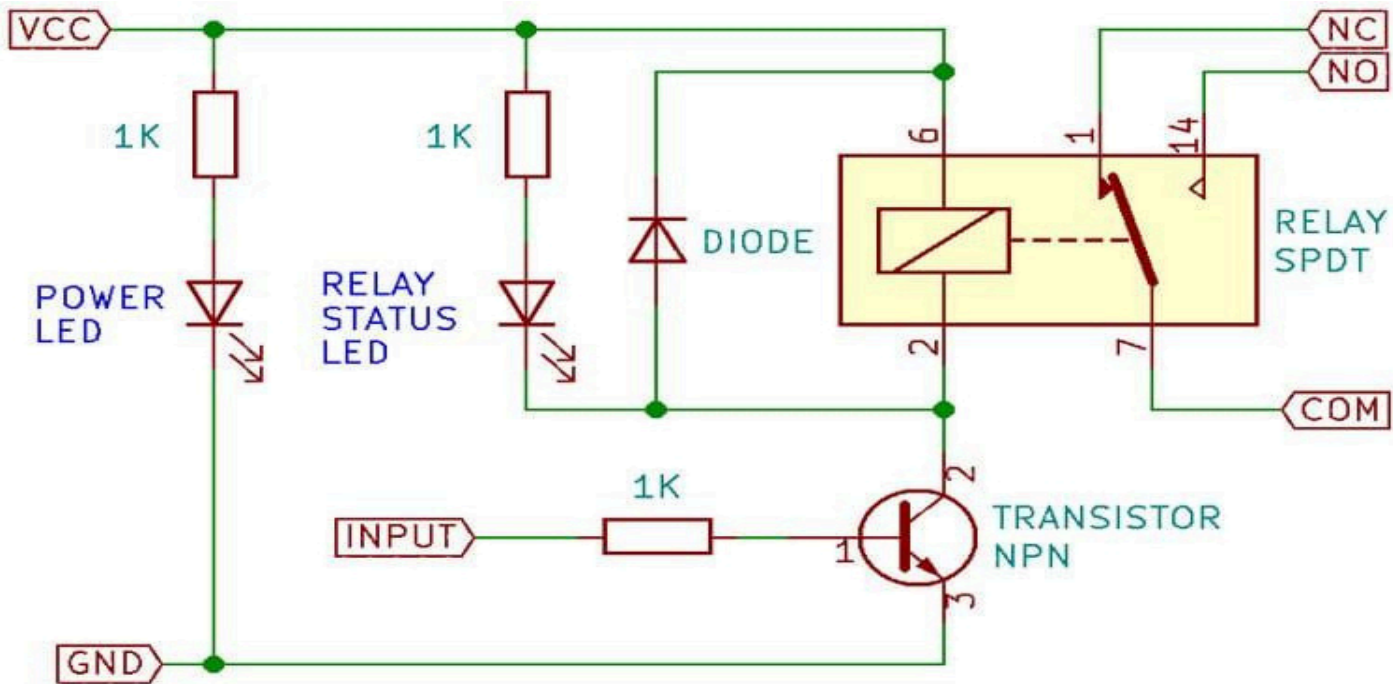
HOW DOES A RELAY WORK?



The relay uses an electric current to open or close the contacts of a switch. This is usually done using the help of a coil that attracts the contacts of a switch and pulls them together when activated, and a spring pushes them apart when the coil is not energized.

There are two advantages of this system – First, the current required to activate the relay is much smaller than the current that relay contacts are capable of switching, and second, the coil and the contacts are galvanically isolated, meaning there is no electrical connection between them. This means that the relay can be used to switch mains current through an isolated low voltage digital system like a microcontroller.

INTERNAL CIRCUIT DIAGRAM FOR SINGLE CHANNEL RELAY



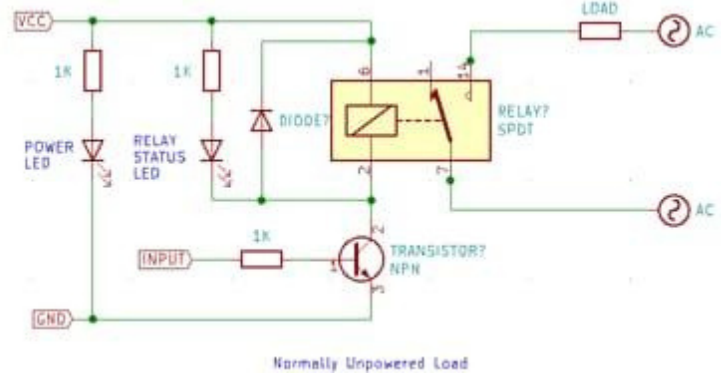
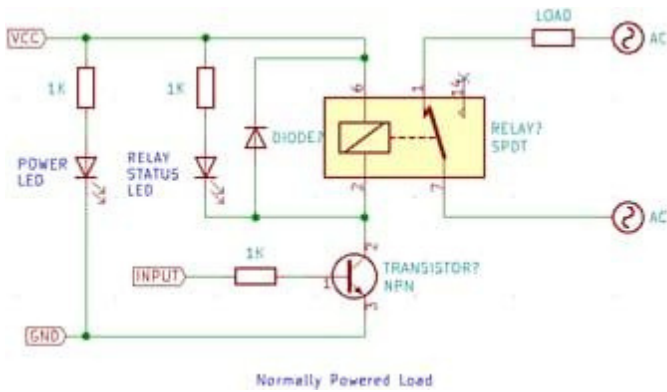
Relay Module Basic Schematic

The extra components apart from the relay are there since it would not be possible to drive the relay directly from the pins of a microcontroller, digital logic or a sensor. This is because although the coil consumes much less current than the currents it can switch, it still needs relatively significant current – low power relays consume around 50mA while higher power relays consume around 500mA. The coil is also an inductive load, so when the coil is switched off, a large flyback voltage is developed which can damage the device turning it on and off. For this reason, a flyback diode is added anti-parallel to the relay coil to clamp the flyback voltage.

LEDs can be added to this basic circuit to act as indicators, and sometimes even optical isolation is added to the input to further improve the isolation

HOW TO USE SINGLE-CHANNEL RELAY MODULE

Relay modules like this one are commonly used to drive mains loads from a microcontroller like the Arduino or a sensor. In cases like this, the common circuit diagram would be as follows.



For simple on/off applications, the relay can be connected as shown above. One terminal of mains is connected to common, and the other is connected to NO or NC depending on whether the load should be connected/disconnected when the relay is active.

BLUETOOTH MODULE(HC-05 Bluetooth Module)

HC-05 Specification:

Bluetooth protocol: Bluetooth Specification v2.0+EDR

Frequency: 2.4GHz ISM band

Modulation: GFSK(Gaussian Frequency Shift

Keying) Emission power: $\leq 4\text{dBm}$, Class 2

Sensitivity: $\leq -84\text{dBm}$ at 0.1% BER

Speed: Asynchronous: 2.1Mbps(Max) / 160 kbps, Synchronous: 1Mbps/1Mbps

Security: Authentication and encryption

Profiles: Bluetooth serial port

Power supply: +3.3VDC 50mA

Working temperature: $-20 \sim +75^\circ\text{C}$

Dimension: 26.9mm x 13mm x 2.2 mm

Overview

HC-05 module is an easy to use Bluetooth SPP (Serial Port Protocol) module, designed for transparent wireless serial connection setup. The HC-05 Bluetooth Module can be used in a Master or Slave configuration, making it a great solution for wireless communication. This serial bluetooth module is fully qualified Bluetooth V2.0+EDR (Enhanced Data Rate) 3Mbps Modulation with complete 2.4GHz radio transceiver and baseband. It uses CSR Bluecore 04- External single chip Bluetooth system with CMOS technology and with AFH (Adaptive Frequency Hopping Feature).

Bluetooth Module HC-05

The Bluetooth module HC-05 is a MASTER/SLAVE module. By default the factory setting is SLAVE. The Role of the module (Master or Slave) can be configured only by AT COMMANDS. The slave modules cannot initiate a connection to another Bluetooth device, but can accept connections. Master module can initiate a connection to other devices. The user can use it simply for a serial port replacement to establish connection between MCU and GPS, PC to your embedded project, etc.

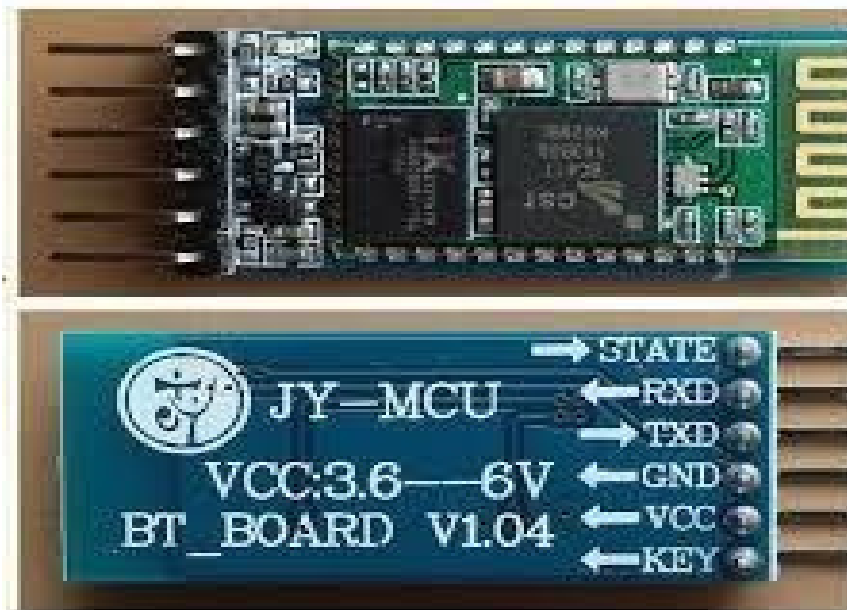
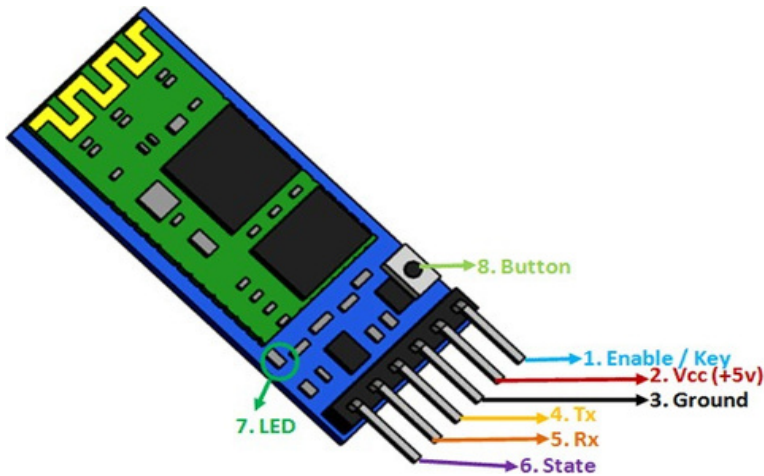


FIG 3: BLUETOOTH MODULE



PIN DESCRIPTION OF BLUETOOTH MODULE

Pin Description:-

The HC-05 Bluetooth Module has 6 pins. They are as follows:

ENABLE:

When enable is pulled LOW, the module is disabled which means the module will not turn on and it fails to communicate. When enable is left open or connected to 3.3V, the module is enabled. The module remains on and communication also takes place.

Vcc:

Supply Voltage 3.3V to 5V

GND:

Ground pin

TXD & RXD:

These two pins act as a UART interface for communication

STATE:

It acts as a status indicator. When the module is not connected to paired with any other bluetooth device, signal goes Low. At this low state, the LED flashes continuously which denotes that the module is not paired with other device. When this module is connected to/paired with

any other bluetooth device, the signal goes High. At this high state, the LED blinks with a constant delay say for example 2s delay which indicates that the module is paired.

BUTTON SWITCH:

This is used to switch the module into AT command mode. To enable AT command mode, press the button switch for a second. With the help of AT commands, the user can change the parameters of this module but only when the module is not paired with any other BT device. The module is connected to any other bluetooth device, it starts to communicate with that device and fails to work in AT command mode.

HC-05 Default Settings:-

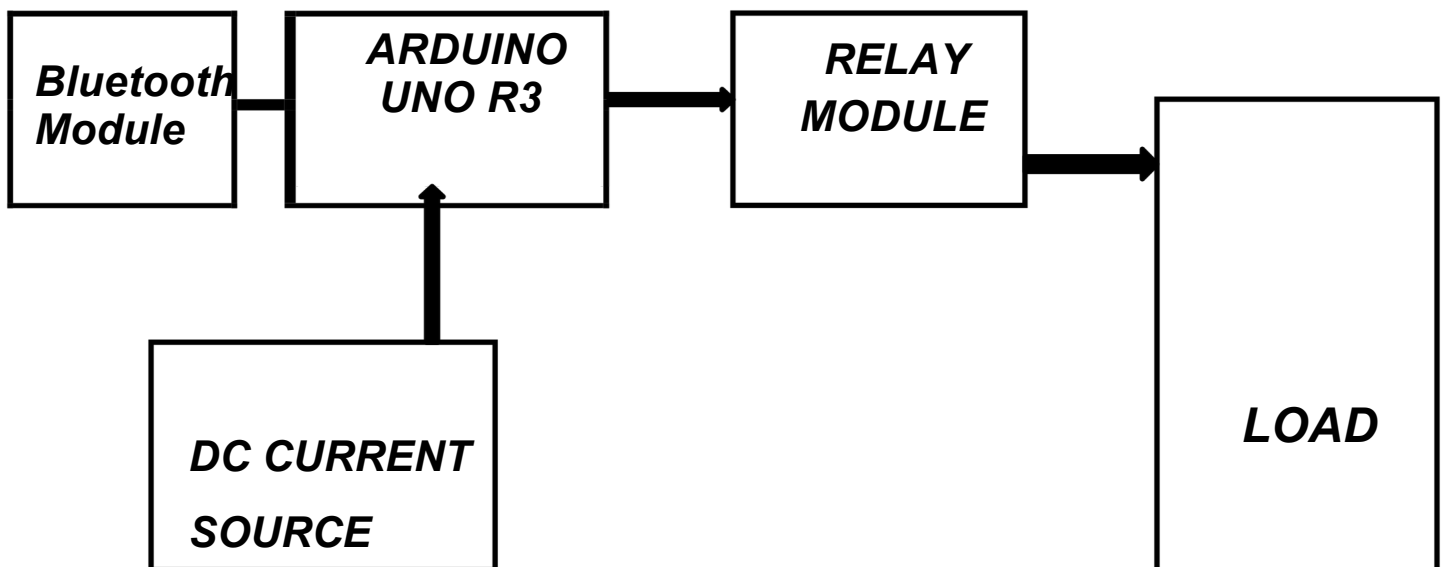
Default Bluetooth Name: ,HC-05' Default Password: 1234 or 0000 Default Communication:

Slave Default Mode: Data Mode

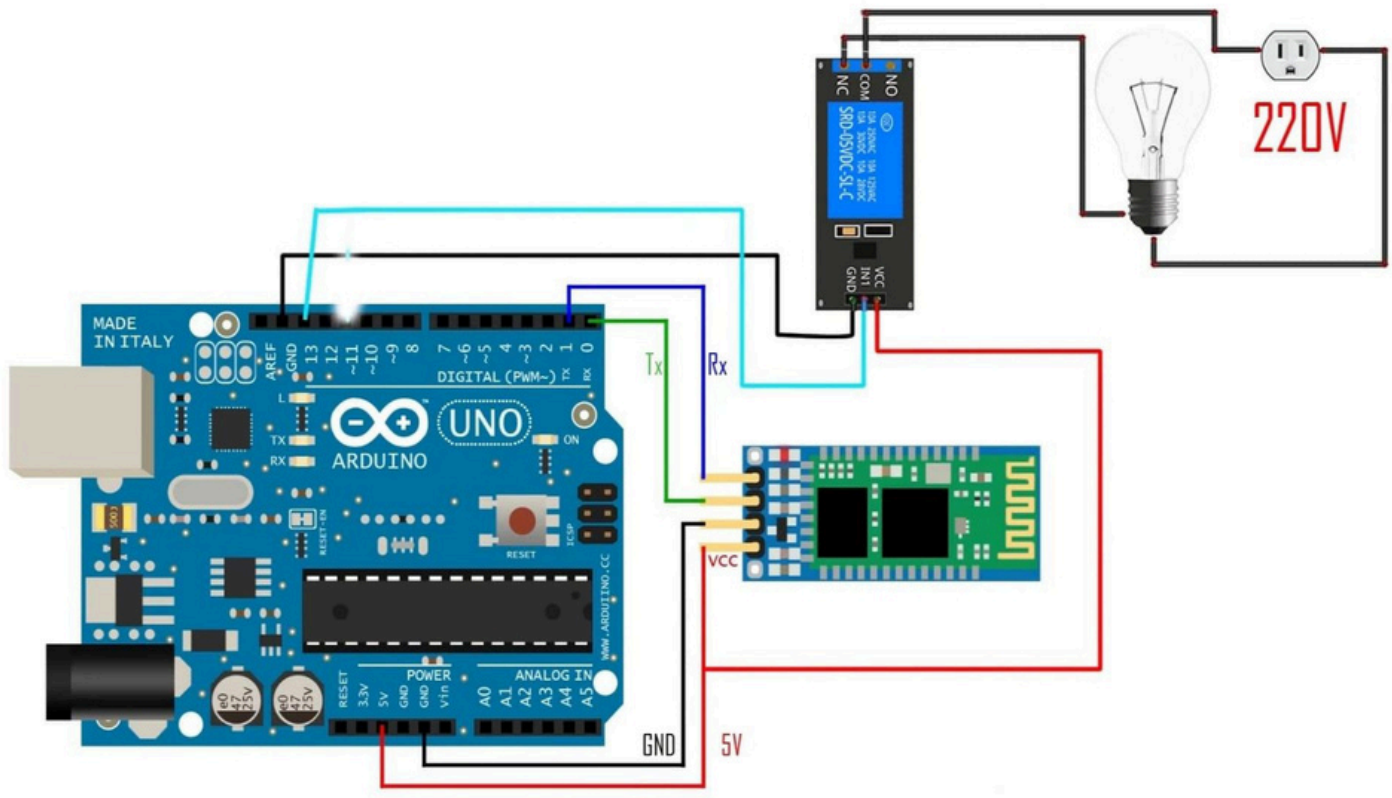
Data Mode Baud Rate: 9600, 8, N, 1

Command Mode Baud Rate: 38400, 8, N, 1 Default firmware: LINVOR

BLOCK DIAGRAM



CIRCUIT DIAGRAM



CIRCUITDIAGRAMOFHOMEAUTOMATIONSYSTEMUSINGARDUINOAND BLUETOOTHMODULE

CODE

```
char val;

void setup() {

  pinMode(13,OUTPUT);
  Serial.begin(9600);
  digitalWrite(13,HIGH);

}

void loop() {

  if(Serial.available()){
    val = Serial.read();
    Serial.println(val);
  }
  if(val=='1'){
    digitalWrite(13,LOW);

  }
  else if(val=='2'){
    digitalWrite(13,HIGH);

  }

  Delay(100)
}
```

APPLICATIONS

Home Is Where the Smart Is

Even machine-to-machine communication, and you understand you're not the most tech-savvy consumer, it's impossible that you've missed the abundance of home automation products filling the shelves and aisles of every home improvement store. Suddenly an ordinary errand for light bulbs will leave you wondering if your lamp could send you a message alerting you that the light bulb needs to be replaced. Furthermore, if your lamp is talking to you, could your refrigerator and sprinkler system be too? Experts say: Yes, the possibilities are endless. If that's the case, where do you begin?

Any day-to-day, repeatable process is automatable with smart home applications. The greater the control and flexibility of these processes, the more energy and cost savings the resident experiences, which are factors anyone who pays utilities strives to moderate. The smart home revolution is likely to be more of an evolution, with the incorporation of one or two home systems at a time, gradually automating our households through smart mobile devices. However, with these elements of efficiency comes the question of ease of use. Will it bring you enjoyment or exasperation? With so many brands and models already available in an ever-growing market, how do you know which is best for you?

Lighting Control: Leaving the Dark Ages and Stepping Into the Light

Smart lighting allows you to control wall switches, blinds, and lamps, but how intuitive is a lighting control system? It turns out, quite; its capabilities are extensive. You're able to schedule the times lights should turn on and off, decide which specific rooms should be illuminated at certain times, select the level of light which should be emitted, and choose how particular lights react through motion sensitivity, as seen with Belkin's WeMo Switch + Motion, which is both affordable and easy to use with its plug-and-play simplicity.

HVAC Regulation: No Longer Burned by Your Heating Bill

As fuel costs rise and the availability and sustainability of our resources becomes a greater concern, heating/cooling our homes efficiently is less a budgetary bonus and more of a necessity. Over the past year, smart thermostats and automated home heating systems have become more readily available and easily incorporate into any home. Heating and cooling our

homes consumes an average of 50% of energy costs yearly, making daily HVAC regulation progressively rewarding. Maintaining a substantial lead among the nearly non-existent

competition, the Nest Learning Thermostat, learns your heating and cooling preferences over time, eliminating the need for programming and is accessible from your smartphone app. With automated HVAC you are able to reduce the heat when a room is unoccupied, and increase or decrease it at specific times based on your schedule and occupancy.

Lawn Irrigation Systems: The Grass is Always Greener

A lush and healthy lawn is a source of pride for most homeowners, but the weather doesn't always cooperate and provide the adequate elements for a flourishing landscape. For decades we've relied on sprinkler systems to keep our yards at peak presentation, but at what cost? The average American home spends approximately 30% of their daily water usage on lawn and garden maintenance. Nearly half of that amount is wasted due to inefficiency. If you apply that statistic to the national average, up to 4.5 billion gallons of water is wasted per day through ineffective watering methods. If we reflect upon the monetary impact of this, it results in Americans spending over a thousand dollars a year in water, with a portion of that being waste. The global effects are even greater when you consider the growing concern over climate change and the dramatic decrease in agricultural natural resources. However, sprinkler control systems, like Skydrop, are providing water regulation through real-time communication with local weather data. If a rain storm develops and deposits two inches of rain water on your lawn, the automated sprinkler detects the saturation and disables its scheduled watering. Conversely, the system will be alerted to dry conditions and supply the necessary amount of nourishment, without over-watering.

Smart Appliances: What's for Dinner?

Will smart kitchen appliances actually make you a better cook? Maybe. Smart refrigerators, such as LG's Smart ThinQ, allow you to scan grocery store receipts and keep an inventory of your items, and alert you if an item is about to expire. More impressively, it suggests recipes based on your refrigerator's contents and lets you know when you need to replace items. Smart ovens sync with your smartphone and automatically preheat to the correct temperature based on a recipe selected from your database. While these appliance options seem a bit superficial and convenience based, there is a conservation factor as well. By automating your kitchen appliance and making them accessible from your smart device, you're able to sever the

electricity supplied to unused appliances and reduce your energy consumption and costs. Considering the number of appliances the average household owns; this could save a substantial amount of money over time.

Security Systems: Knock, Knock...

Who's there? The Internet of Things. While efficiency and conservation are certainly IoT benefits, its potential to have improved control over home security is a primary focus. Smart locks, like Kwikset's Kevo, a Bluetooth enabled electronic deadbolt, and various connected home security systems, such as iSmartAlarm, offer a variety of features including door and window sensors, motion detectors, video cameras and recording mechanisms. All of which are connected to a mobile device and accessible via the cloud, thus enabling you to access real-time information on the security status of your home. Naturally, there is a great deal of scrutiny regarding the level of trust in controlling your home's security system via a mobile device, but it begs earnest exploration when weighing the potential benefits and peace of mind it provides homeowners.

CONCLUSION

The system as the name indicates, 'Home automation' makes the system more flexible and we integrate mobile devices into home automation systems. A novel architecture for a home automation system is proposed using the relatively new communication technologies. The system consists of mainly three components is a BLUETOOTH module, Arduino microcontroller and relay circuits. WIFI is used as the communication channel between android phone and the Arduino microcontroller. We hide the complexity of the notions involved in the home automation system by including them into a simple, but comprehensive set of related concepts. This simplification is needed to fit as much of the functionality on the limited space offered by a mobile device's display. This paper proposes a low cost, secure, ubiquitously is novel and has achieved the target to control home appliances remotely using the WiFi technology to connects system parts, satisfying user needs and requirements. WiFi technology capable solution has proved to be controlled remotely, provide home security and is cost- effective as compared to the previously existing systems. Hence we can conclude that the required goals and objectives of home automation system have been achieved. The system design and architecture were discussed, and prototype presents the basic level of home appliance control and remote system monitoring has been implemented. Finally, the proposed is better from the scalability and flexibility point of view than the commercially available home automations systems.